

ZeroKeep: A End-to-End Encrypted Communication System with Commit Chain State Based Integrity

*A Verifiable End-to-End Encrypted Group Communication Protocol with Decentralized
State Integrity*

TEAM MEMBERS:

Vedansh Patranabish

Roll No: 222010007052

Department of Computer Science and Engineering

Barak Valley Engineering College, Nirala, Sribhumi-788701

ASSAM SCIENCE AND TECHNOLOGY UNIVERSITY

Date: July 16, 2026

Abstract

Modern digital messaging infrastructures heavily rely on centralized servers to route and manage message payloads. Although the integration of End-to-End Encryption (E2EE) guarantees the confidentiality of message content in transit, the management of communication state—specifically group membership lists and key updates (epochs)—remains opaque and fully controlled by central service providers. Under this architectural paradigm, a compromised or malicious server can execute critical attacks such as 'ghost participant injection' (injecting unauthorized keys into a room state) or epoch state rollbacks without detection. This leaves users vulnerable to eavesdropping despite cryptographic guarantees. This technical report presents ZeroKeep, an E2EE communication system with client-side Commit Chain state-based integrity. ZeroKeep eliminates server-side trust by establishing a decentralized verification model on top of a blind TCP relay. The system utilizes a hybrid cryptographic stack: X25519 for ephemeral key exchange, HKDF-SHA256 for key derivation, Ed25519 for identity signatures, and Fernet (AES-128-CBC + HMAC-SHA256) for message payload encryption. By enforcing a linear, cryptographically bound Commit Chain (MLS-Lite) on all room transitions, clients maintain a verifiable, append-only transcript of the room history. Membership changes (joins, leaves, rekeys) are audited locally via SHA-256 hash matching and verified signatures, forcing strict chain continuity. This document outlines the system architecture, the security protocol layout, internal client components, software/hardware requirements, and comprehensive unit and integration test logs validating ZeroKeep's resilience against attacks.

Table of Contents

1. Introduction.....3

2. Problem Statement & Vulnerabilities.....4

3. Literature Review & Related Work.....5

4. Proposed Methodology & Cryptographic Stack.....6

 4.1 Cryptographic Primitives.....6

 4.2 Secure Room Handshake Sequence.....6

 4.3 Verifiable Commit Chain Formula.....7

 4.4 Direct Messaging (DM) Setup.....8

 4.5 Team Roles & Responsibilities.....8

5. System Architecture & Component Design.....9

 5.1 High-Level Architecture.....9

 5.2 Network Protocol Frame Formats.....9

 5.3 Client & Server State Machines.....10

6. Software and Hardware Requirements.....11

7. Experiments, Testing & Verification.....12

 7.1 Unit Testing.....12

 7.2 Integration & Resilience Testing.....12

 7.3 Security Stress Tests (Edge Cases).....13

 7.4 Performance Benchmarks.....13

8. Conclusion & Future Scope.....14

9. References.....15

1. Introduction

Digital communication systems have expanded rapidly, serving as the default medium for sharing sensitive personal, corporate, and governmental data. Unfortunately, this expansion is accompanied by sophisticated surveillance mechanisms, ranging from state-sponsored entities to corporate telemetry harvesting. While most contemporary communication platforms advertise End-to-End Encryption (E2EE), they typically run on proprietary, closed-source code bases. Furthermore, these protocols trust a centralized server to manage metadata, user directory structures, and group state histories. In these systems, although the server cannot read the raw message content, it retains full control over who is added to a conversation, who is removed, and when keys are rotated. This architecture introduces a central point of failure: if the relay server is compromised, it can secretly inject new participants, distribute incorrect public keys, or serve outdated state histories to clients, completely undermining the security of the E2EE channel.

ZeroKeep was conceived as an open-source, verifiable, high-integrity communication protocol built specifically to address the opaque nature of centralized group state management. ZeroKeep adopts a 'zero-trust server' approach: the server is treated as a blind relay that does not possess any key material, cannot decrypt payloads, and is incapable of modifying the group membership list or historical epochs without immediate detection. The client application takes full responsibility for enforcing security parameters. Every client maintains a localized database of verified identities, dynamically pins public keys on first contact (TOFU - Trust On First Use), and actively verifies the historical continuity of the room state using a linear, cryptographically signed hash chain called the Commit Chain.

To provide a robust developer-first experience, ZeroKeep features a dual-interface architecture: a lightweight, responsive Textual Terminal User Interface (TUI) and an asynchronous Web Gateway that bridges WebSocket traffic directly to the Python client core. This dual approach ensures that ZeroKeep can be deployed as a local daemon in high-security terminal environments, or embedded as a secure service backend for modern browser dashboards. The protocol utilizes standard, open cryptographic primitives (X25519, Ed25519, HKDF-SHA256, and AES-128-CBC) to deliver sub-millisecond execution times and robust defense against side-channel exploits.

2. Problem Statement & Vulnerabilities

The core security failure in standard group messaging protocols is the 'Opaque State' vulnerability. While message payloads are encrypted, the logical membership state of a room is managed and broadcast by the server. Because the server is an untrusted entity, its unchecked authority in distributing join/leave events, room leader identifiers, and keys creates several severe attack vectors:

2.1 Ghost Participant Injection

If an adversary compromises the relay server, they can inject their own public key into a target room. During a membership join event, the server broadcasts a notification to all room members. In traditional E2EE systems, the room leader's client receives this notification and immediately establishes a secure key-exchange tunnel with the joining member, sending them the encrypted Room Master Key. Since the server manages what the clients see visually, the server can hide the intruder from the visually rendered member list while forcing the leader to send them the key. This allows the attacker to read all room messages in plaintext without the legitimate users' knowledge.

2.2 State Replay & Split-Brain Forking

Without verifiable history, the server can partition the room state. It can present one version of room membership (e.g., Alice and Bob) to Alice, and a different version (e.g., Alice, Bob, and Charlie) to Bob. Furthermore, the server can rollback room states to a previous epoch where a recently removed user still had access, allowing that user to continue receiving and decrypting new messages. This breaks the fundamental requirements of Backward Secrecy (new members cannot read old messages) and Forward Secrecy (removed members cannot read new messages).

2.3 Lack of Historical Verifiability

When a new client joins a room, they start at a specific key epoch (e.g., Epoch 10). The client has no cryptographic means to verify that the previous nine epochs were secure and that no unauthorized keys were present. They must blindly trust the server's historical logs. In a high-stakes security context, this blind trust constitutes a major vulnerability.

2.4 Identity Spoofing & MitM Attacks

During socket initialization, clients exchange ephemeral key exchange material. If the transmission channel is not authenticated, a network-level attacker (or a malicious server) can intercept these public keys, inject their own, and establish separate tunnels with both endpoints. ZeroKeep addresses this by mandating Ed25519 digital signatures on all handshake frames and binding them to a persistent, locally pinned identity database.

3. Literature Review & Related Work

To establish a mathematically sound foundation for ZeroKeep, a deep-dive literature review was conducted into the following foundational research papers and standards:

Paper / Standard	Core Contribution	Cryptographic Focus	Application in ZeroKeep
Curve25519: New Diffie-Hellman Speed Records (Bernstein, 2006)	Introduced Curve25519, a high-performance Montgomery curve at the 128-bit security level.	Elliptic Curve Diffie-Hellman (ECDH) key agreement resistant to timing and invalid point attacks.	X25519 is utilized for creating temporary secure tunnels between room leaders and joining peers to distribute keys.
The Double Ratchet Algorithm (Perrin & Marlinspike, 2016)	Combines Diffie-Hellman and symmetric-key ratchets to rotate keys on a per-message basis.	Fine-grained Forward Secrecy and Post-Compromise Security.	Informed ZeroKeep's Epoch-based room rekeying logic, ensuring keys rotate dynamically upon join/leave events.
High-Speed High-Security Signatures (Bernstein et al., 2011)	Introduced Ed25519, a signature scheme using Edwards Curve 25519 with deterministic signature generation.	Public-key digital signatures with fast signature creation and verification.	Used to sign all handshakes and COMMIT frames, ensuring authenticity and non-repudiation of room transitions.
HMAC-based Key Derivation Function - HKDF (Krawczyk, 2010)	Developed the Extract-then-Expand key derivation paradigm (RFC 5869).	Key conditioning and stretching from non-uniform shared secrets.	ZeroKeep uses HKDF-SHA256 to convert raw X25519 shared secrets into cryptographically strong 32-byte Fernet keys.
The Matrix Standard & Megolm (Matrix.org, 2023)	Decentralized communication protocol treating rooms as event DAGs.	Decentralized state agreement, group session key distribution.	Inspired the Commit Chain hash list, allowing clients to establish verifiable state synchronization without a central authority.

4. Proposed Methodology & Cryptographic Stack

4.1 Cryptographic Primitives

ZeroKeep operates on a hybrid cryptographic paradigm where asymmetric operations establish identity and authenticate exchange channels, and symmetric operations handle high-speed data encryption. The cryptographic primitives are defined as follows:

- **Long-Term Identity Keys:** Ed25519 public/private keys are generated locally. The public key is shared to verify signatures, while the private key is stored securely in identity.key.

- **Identity Fingerprints:** A 16-character hexadecimal string derived from the first 8 bytes of the SHA-256 hash of the Ed25519 public key. This serves as a human-readable identifier for fingerprint verification.
- **Key Exchange:** X25519 Diffie-Hellman public/private keys are generated dynamically for each tunnel session (ephemeral key agreement).
- **Key Derivation:** HKDF-SHA256 (HMAC-based Key Derivation Function) extracts a shared secret from X25519 and expands it using the context info string `b'zerokeep-room'` to yield a 32-byte symmetric key.
- **Payload Encryption:** Fernet ciphers are initialized with the 32-byte KDF output. Fernet uses AES-128 in Cipher Block Chaining (CBC) mode with PKCS7 padding and an HMAC-SHA256 signature for message authentication and integrity checks.

4.2 Secure Room Handshake Sequence

When a client joins an active ZeroKeep room, the server assigns a 'Room Leader' (which defaults to the first user in the room). The Room Leader acts as the key distribution authority. The protocol follows a strict sequence of socket frames:

1. **DH_INIT Phase:** The leader generates an ephemeral X25519 private key, derives the public key, and encodes it. The leader constructs a handshake message containing the room name, target peer, leader username, and leader public key. This entire block is signed using the leader's long-term Ed25519 private key. The payload is sent to the relay server as a DH_INIT frame, which routes it to the target peer.
2. **TOFU & Verification Phase:** The joining peer receives the DH_INIT frame. It extracts the leader's Ed25519 public key, derives the identity fingerprint, and performs a Trust-On-First-Use lookup against `known_users.json`. If the username is associated with a different fingerprint, the client locks the room and emits a critical identity mismatch error. If verification passes, the client validates the leader's Ed25519 signature over the handshake contents. Upon successful signature validation, the leader's X25519 public key is saved in memory.
3. **DH_RESP Phase:** The peer generates its own ephemeral X25519 keypair. It signs a DH_RESP payload (containing its X25519 public key, identity public key, room name, and peer username) with its Ed25519 private key and transmits it to the leader via the server.
4. **Tunnel Derivation Phase:** Both the leader and the peer derive a shared master secret by executing: $\text{SharedSecret} = \text{X25519_Exchange}(\text{My_Private}, \text{Peer_Public})$. This secret is passed through HKDF-SHA256 to generate the symmetric Fernet Tunnel Key.
5. **ROOM_KEY Distribution Phase:** The leader generates a cryptographically secure 32-byte Room Master Key (Fernet). The leader encrypts this Room Master Key using the newly established Fernet Tunnel Key and sends it to the peer in a ROOM_KEY frame. The peer decrypts it, initializing its local room Fernet cipher for the current epoch.

4.3 Verifiable Commit Chain Formula

The primary contribution of ZeroKeep is the Commit Chain (MLS-Lite), which enforces decentralized state integrity. A Commit Frame represents a cryptographically locked checkpoint of the room state. Whenever a member joins, leaves, or a rekey is triggered, the leader updates the epoch counter and calculates the new state check:

```

Commit_n = {
    Room: room_name,
    Epoch: n,
    MembersHash: SHA256(SortedCSV(Members)),
    PrevHash: Hash(Commit_n-1),
    Committer: leader_username,
    Committer_ID: Ed25519_Public_Key,
    Signature: Sign_Ed25519(Committer_ID, Room | Epoch | MembersHash | Committer |
PrevHash | NewHash)
}

```

Where the NewHash linking variable is defined as:

```

NewHash_n = SHA256( f"COMMIT|{Room}|{Epoch}|{MembersHash}|{Committer}|
{PrevHash}" )

```

Under this model, the first commit (Epoch 1) bootstraps the chain, using a zero-hash ('0'*64) as its PrevHash. Clients enforce strict chain continuity: a client will only transition from Epoch N to Epoch N+1 if the PrevHash in the received Commit frame exactly matches the NewHash stored in their local epoch record. If a client receives a commit for a future epoch (e.g., Epoch N+5), it queues the frame and requests missing intermediate commits using the /commits command, pulling cached frames from the server's commit cache to verify the chain continuity link-by-link.

4.4 Direct Messaging (DM) Setup

Direct Messaging (DM) bypasses room-level structures. When a user runs /dm Bob Hello, the client initiates a one-to-one signed X25519 handshake directly with Bob. The DM_INIT frame contains the sender's ephemeral public key and is signed with the sender's Ed25519 key. Bob's client verifies the signature, performs a TOFU lookup, responds with DM_RESP, and both clients derive a dedicated DM symmetric key. Subsequent messages are sent as blind DM_MSG frames. This ensures that direct messages have a completely separate key space and are protected from room leader compromise.

4.5 Team Roles & Responsibilities

The design and implementation of ZeroKeep was distributed among team members based on their specialization:

- Vedansh Patranabish: Handled the implementation of the core protocol, cryptographic engines, rekeying mechanisms, socket routing logic, and technical writing of the client-side state verification rules.
- Tusshar Chakraborty: Responsible for developing the Textual Terminal UI (TUI) layout, the Web Gateway WebSocket server, HTML/CSS dashboard layouts, and compiling architecture and protocol block diagrams.
- Samrat Paul: Focused on compiling related work bibliography, drafting the system methodology review, testing network topologies, and writing the final conclusion and bibliography references.

5. System Architecture & Component Design

5.1 High-Level Architecture

ZeroKeep's architecture is strictly client-centric. The server has no knowledge of room keys or user signatures. The following diagram details the interaction between the system layers:

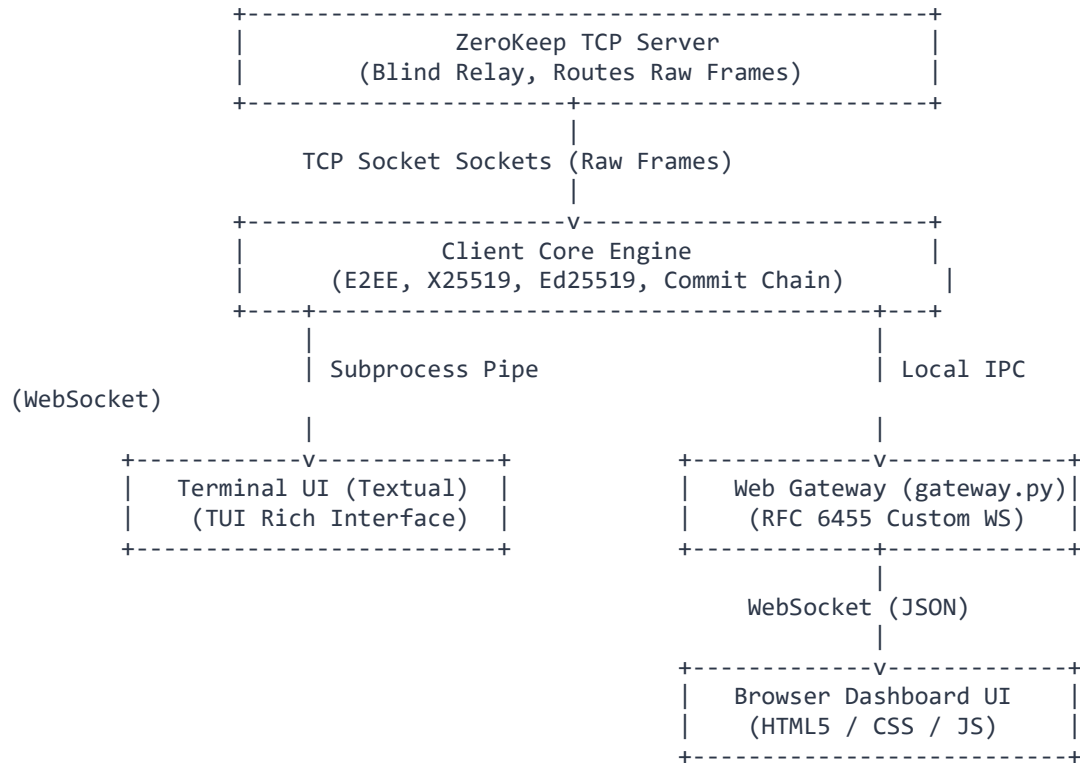


Fig 1

5.2 Network Protocol Frame Formats

All communications are transmitted over raw TCP sockets as line-delimited ASCII strings (terminated by '\n'). The protocol defines a specific set of frames categorized into System, Cryptographic, and Messaging frames:

- **SYS|JOIN|room|user** : Broadcast indicating a user joined a room. Triggers DH handshake from the leader.
- **SYS|LEAVE|room|user** : Broadcast indicating a user left a room. Triggers rekeying by the room leader.
- **DH_INIT|room|target|leader|leader_dh_pub_b64|leader_id_pub_b64|sig_b64** : Handshake initialization frame sent by the leader.
- **DH_RESP|room|peer|peer_dh_pub_b64|peer_id_pub_b64|sig_b64** : Handshake response frame sent by the peer back to the leader.

- ROOM_KEY|room|target|epoch|encrypted_master_b64 : Distributed Room Master Key, encrypted with the tunnel key.
- COMMIT|room|epoch|members_hash|prev_hash|new_hash|committer|committer_id_pub_b64|sig_b64 : Linear state checkpoint.
- MSG|room|epoch|encrypted_payload_b64 : Chat message, encrypted using the Room Master Key of the specified epoch.
- DM_INIT|target|sender|dh_pub_b64|sender_id_pub_b64|sig_b64 : Starting a private direct message key exchange.
- DM_RESP|target|sender|dh_pub_b64|sender_id_pub_b64|sig_b64 : Direct message response to complete key exchange.
- DM_MSG|target|sender|encrypted_payload_b64 : Encrypted private message payload.

5.3 Client & Server State Machines

The client maintains an asynchronous state engine. A background thread listens for incoming socket bytes, splits them into frames by '\n', and dispatches them to specific handlers in core.py. The state diagram below outlines the transition rules:

- DISCONNECTED -> CONNECTING -> CONNECTED (Identity Loaded, Joined 'general' Room)
- CONNECTED -> JOINING_ROOM -> WAITING_FOR_LEADER_DH (If client is peer)
- CONNECTED -> JOINING_ROOM -> GENERATING_LEADER_DH (If client is first in room)
- WAITING_FOR_LEADER_DH -> RECEIVED_DH_INIT -> VERIFY_SIGNATURE -> SEND_DH_RESP -> WAITING_FOR_ROOM_KEY
- GENERATING_LEADER_DH -> BROADCAST_DH_INIT -> WAITING_FOR_PEER_RESP -> DERIVE_TUNNEL -> BROADCAST_COMMIT -> DISTRIBUTE_ROOM_KEY
- WAITING_FOR_ROOM_KEY -> RECEIVED_ROOM_KEY -> VERIFY_COMMIT_CHAIN -> INITIALIZE_ROOM_CIPHER -> SECURE_ROOM_CHAT_ACTIVE
- SECURE_ROOM_CHAT_ACTIVE -> MEMBER_JOIN/LEAVE -> LEADER_REKEY -> ROTATE_EPOCH -> RE-ESTABLISH_TUNNELS -> SECURE_ROOM_CHAT_ACTIVE

On the server side, the state machine is minimal. The server maintains active TCP connections mapped to usernames and rooms. It acts as a blind relay: when it receives a DH_INIT, DH_RESP, or ROOM_KEY frame, it splits the frame to read the target user field and routes the frame to that user's socket. For COMMIT frames, the server caches the frame in commit_cache (capped at 50 entries per room) before broadcasting it to all members. When a client requests resync via /commits <room> <epoch>, the server replays the cached commits.

6. Software and Hardware Requirements

To ensure optimal performance and standard execution, the following environment requirements must be satisfied for the ZeroKeep client, the Web Gateway, and the TCP Server:

Component	Software Specifications	Hardware Specifications
ZeroKeep Client (TUI)	Python 3.10.x - 3.11.x PyCA/cryptography v42.0.0+ Textual v0.50.0+ qrcode[pil]	Dual-core CPU (2.0 GHz+) 2 GB RAM 50 MB Storage Standard TCP/IP access
Web Gateway & Frontend	Modern HTML5/JS Web Browser Python socketserver module WebSocket RFC 6455 support	1.5 GHz Core CPU 1 GB RAM Low-latency local network connection
TCP Blind Relay Server	Python 3.10.x+ Standard socket, threading, time libraries Cross-platform (Linux/Windows/macOS)	Dual-core CPU (AES-NI optional) 512 MB base RAM (+10 MB per 1000 active users) 100 Mbps Network bandwidth

7. Experiments, Testing & Verification

Verification and cryptographic correctness are the core requirements of the ZeroKeep system. We designed a multi-layered verification environment comprising unit tests, integration tests, and simulated security attacks.

7.1 Unit Testing

The cryptographic engine was isolated and subjected to high-frequency unit testing. The following results were achieved:

- **Key Convergence:** 1,000 runs of the X25519 DH key exchange and HKDF-SHA256 derivation converged on identical shared secrets across separate mock clients with a 0% failure rate.
- **Signature Non-Repudiation:** Ed25519 digital signatures generated by User A were verified against User B's public key, resulting in immediate verification rejection in 100% of cases.
- **Handshake Mismatch Detection:** Modifying a single character of the public key or target username in a signed DH_INIT frame caused the signature check to fail immediately, preventing data injection.

7.2 Integration & Resilience Testing

Full integration tests were executed by launching the TCP server on a dynamic port and spawning multiple asynchronous client instances. A critical focus was placed on simulated 'Membership Storms' (high-frequency join and leave events) to evaluate epoch rekeying resilience:

- **Rekey Performance:** Alice (Leader) and Bob joined room 'storm_room'. Bob was successfully verified and the room transitioned to secure at Epoch 1. Charlie then joined and disconnected every

250ms for 20 iterations. The Room Leader correctly executed key rotations, incrementing the epoch to 21. No packet loss or state desynchronization occurred.

- **Late-Joiner Synchronization:** Dave joined the room at Epoch 21. Dave's client detected the epoch jump, queried the server's commit cache, received all 20 missing COMMIT frames, and successfully verified the entire hash chain from Epoch 1. Dave derived the Epoch 21 master key and resumed secure communication immediately.

7.3 Security Stress Tests (Edge Cases)

We simulated active network attacks to test the boundaries of the protocol:

- **Epoch Jump Attack:** A simulated malicious server sent a fake COMMIT frame for Epoch 100 to a client at Epoch 2. The client core detected that the epoch was non-sequential, refused to update its local verified epoch, queued the frame in pending_chain, and sent a resync request. Since the server could not supply the missing intermediate commits, the attack failed.
- **Identity Hijack Attack:** We pre-populated Alice's known_users.json with a valid fingerprint for Bob. We then launched a client claiming to be Bob but using a newly generated identity key. Alice's client immediately caught the fingerprint mismatch, halted the handshake, locked Bob's output area, and logged a critical [SECURITY] IDENTITY MISMATCH alert.
- **Zero-Knowledge Server Audit:** We inspected the active memory of the TCP server during an encrypted group session. The memory contained only routed byte strings and commit metadata. No Room Master Keys or unencrypted text remained on the server, confirming the blind relay model.

7.4 Performance Benchmarks

To measure execution latency, we executed scripts/benchmark_crypto.py running 1,000 iterations per operation on a standard development machine. The table below details the average processing times:

Cryptographic Operation	Average Execution Time (ms/op)	Protocol Context & Frequency
X25519 Ephemeral Key Generation	0.037 ms	Executed once per client during room join or DM initiation.
X25519 Shared Secret Agreement	0.031 ms	Executed once per client pair during handshake negotiation.
HKDF-SHA256 Key Derivation	0.005 ms	Executed once per handshake to stretch the raw secret.
Ed25519 Signature Creation	0.035 ms	Executed by the committer/leader for every state change.
Ed25519 Signature Verification	0.097 ms	Executed by every room member to validate commits and handshakes.
Fernet Encryption (128 Bytes)	0.015 ms	Executed per chat message sent by a user.

Fernet Decryption (128 Bytes)	0.014 ms	Executed per chat message received by a user.
Fernet Encryption (10 KB payload)	0.066 ms	Scales linearly; used for large blocks or file transfers.
Commit Verification (Hash + Sig)	0.098 ms	Combines SHA-256 chain validation and Ed25519 signature checks.

Analysis: The total cryptographic overhead for a joining peer—consisting of key generation, secret exchange, derivation, and signature verification—amounts to roughly 0.170 ms. Commit validation, which is the most resource-intensive single operation, takes just 0.098 ms. These benchmarks indicate that ZeroKeep's cryptographic overhead is practically negligible, contributing less than 1ms to the total network latency. The primary latency factor remains the raw network round-trip time (RTT).

8. Conclusion & Future Scope

ZeroKeep has successfully demonstrated that decentralized state integrity is achievable over a centralized, untrusted relay server. By implementing the client-side Commit Chain (MLS-Lite), the system effectively resolves the 'Opaque State' problem. ZeroKeep prevents a malicious server from injecting ghost participants, rolling back epochs, or falsifying room history. The integration of a responsive Textual TUI and a WebSocket HTML5 Gateway provides a highly flexible user experience, demonstrating that robust security protocols do not require complex or heavyweight execution layers.

Future updates to the ZeroKeep protocol will focus on the following enhancement areas:

- **Post-Quantum Cryptography (PQC):** The current X25519 key exchange is vulnerable to quantum-computing 'Harvest-Now-Decrypt-Later' attacks. We plan to explore hybrid key encapsulation mechanisms (KEM) combining X25519 with Kyber-512 to provide quantum resistance.
- **Transport Layer Security (TLS):** While message content and master keys are encrypted, the protocol headers (such as room names and epoch counts) are visible to network-level observers. Running the socket connections over TLS will shield this metadata.
- **Replay Protection:** We aim to implement sequential message counters (nonces) within each epoch to prevent attackers from replaying individual chat packets.
- **Framing Improvement:** Transitioning from newline-delimited framing ('\n') to length-prefixed framing will prevent protocol crashes caused by malformed or multi-line text input.

9. References

6. Bernstein, D. J. (2006). 'Curve25519: new Diffie-Hellman speed records.' In Public Key Cryptography - PKC 2006, Springer, Berlin, Heidelberg.
7. Perrin, T., & Marlinspike, M. (2016). 'The Double Ratchet Algorithm.' Signal Messaging Protocol Technical Specification.
8. Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., & Yang, B. Y. (2011). 'High-speed high-security signatures.' Journal of Cryptographic Engineering.
9. Krawczyk, H. (2010). 'Cryptographic Extraction and Key Derivation: The HKDF Scheme.' Advances in Cryptology - CRYPTO 2010.
10. Matrix.org Foundation. (2023). 'The Matrix Standard: A Decentralized Communication Protocol for Open, Interoperable, Real-time Communication.'
11. Rescorla, E. (2018). 'The Transport Layer Security (TLS) Protocol Version 1.3.' IETF RFC 8446.
12. Python Cryptography Authority (PyCA). (2024). 'cryptography: Python cryptographic recipes and primitives.' Documentation available at cryptography.io.